

Product Requirements Document (PRD)

White-Label SaaS Platform with Subdomain + Custom Domain Support For Node.js + NGINX on VPS (Hostinger)

1. Product Overview

You are building a **multi-tenant white-label SaaS platform** hosted centrally on your own VPS infrastructure.

Example:

- Main Platform:
syndicationx.com
- Tenant Subdomain:
amazon.syndicationx.com
- Tenant Custom Domain:
amazonmarketing.com

All tenants use the **same centralized Node.js application**, but each tenant experiences:

- Their own branding
- Their own domain/subdomain
- Their own data isolation
- Their own SSL certificate
- Their own configurations

The source code is NEVER shared with customers.

2. Core Business Goals

Primary Goals

2.1 White-Label SaaS

Allow clients to use:

- platform subdomain
- OR custom domain

without exposing infrastructure or code.

2.2 Centralized Deployment

Maintain:

- single Node.js application
 - single deployment pipeline
 - centralized updates
 - centralized security patches
-

2.3 Dynamic Tenant Routing

Automatically identify tenant by:

- subdomain
 - custom domain
 - request hostname
-

2.4 Enterprise-grade SSL

Support:

- wildcard SSL
 - automatic Let's Encrypt SSL
 - zero downtime renewals
-

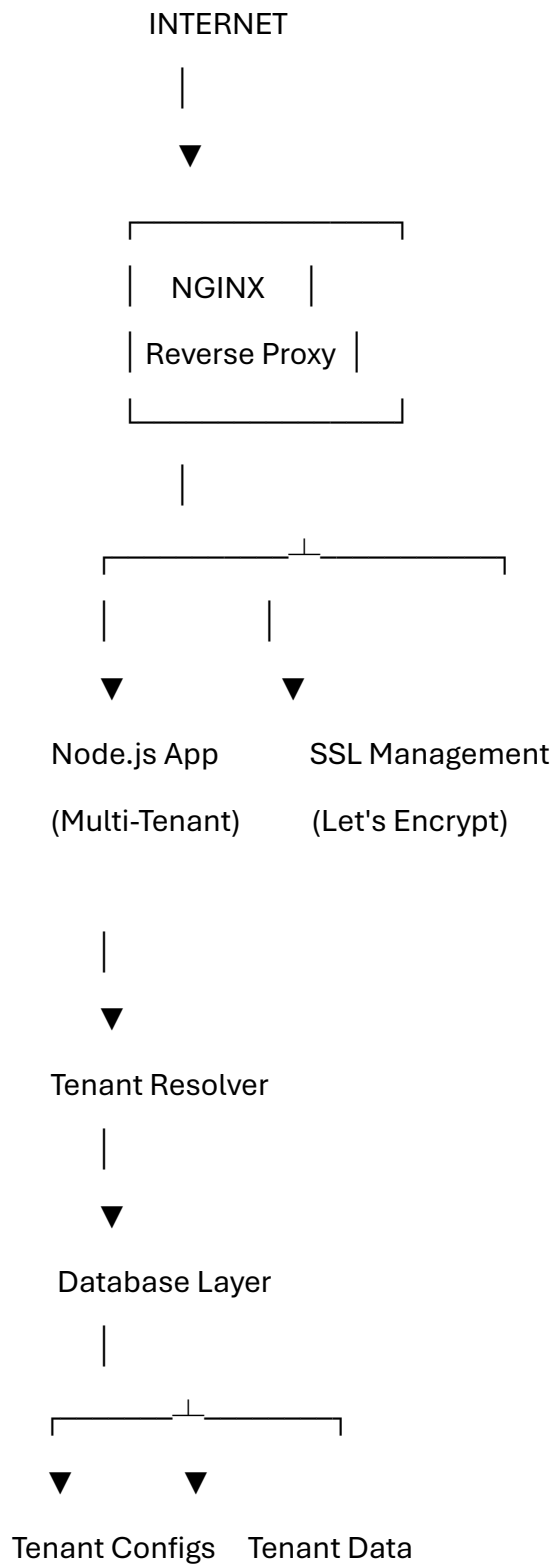
2.5 Scalable Architecture

Support future scaling:

- 10 tenants
- 100 tenants
- 10,000 tenants

without architectural rewrite.

3. High-Level Architecture



4. Functional Requirements

4.1 Tenant Identification

The system MUST identify tenants dynamically using:

Type	Example
Subdomain	amazon.syndicationx.com
Custom Domain	amazonmarketing.com

Logic

Request arrives:

Host Header:

amazon.syndicationx.com

OR

Host Header:

amazonmarketing.com

Node.js resolves:

tenant = findTenantByDomain(hostname)

4.2 Tenant Database Schema

tenants table

Field	Type
id	UUID
name	VARCHAR
subdomain	VARCHAR
custom_domain	VARCHAR
ssl_status	ENUM
status	ENUM
branding_config	JSON
created_at	TIMESTAMP

4.3 Tenant Branding

Each tenant may configure:

- logo
- colors
- favicon
- email templates
- SMTP
- custom CSS
- analytics keys

Stored in DB or object storage.

4.4 Custom Domain Support

Customers can add:

- apex domains
- www domains
- subdomains

Examples:

- amazonmarketing.com
 - www.amazonmarketing.com
 - app.amazonmarketing.com
-

4.5 DNS Verification

System must verify domain ownership.

Supported Verification

Option 1 — CNAME

app.customer.com

CNAME

proxy.syndicationx.com

Option 2 — A Record

customer.com

A

YOUR_VPS_IP

4.6 SSL Automation

System should:

- auto issue SSL
- auto renew SSL
- auto attach SSL to NGINX

Recommended:

- Let's Encrypt
 - Certbot
 - acme.sh
-

4.7 Admin Dashboard

Internal admin features:

Feature	Description
Add tenant	Create new tenant
Domain mapping	Attach custom domain
SSL status	Monitor certificates
Suspend tenant	Disable tenant
Branding	Manage white-label settings
Usage analytics	Tenant traffic

5. Technical Requirements

5.1 Backend Stack

Component	Technology
Runtime	Node.js
Framework	Express / Fastify / NestJS
Reverse Proxy	NGINX
SSL	Let's Encrypt
DB	PostgreSQL
Cache	Redis
Process Manager	PM2
Object Storage	S3 / MinIO

5.2 Multi-Tenant Strategy

Recommended:

Shared Application + Shared Database + Tenant Isolation

Use:

tenant_id

on every business table.

Example:

users

orders

products

campaigns

5.3 Middleware Tenant Resolver

Example:

```
app.use(async (req, res, next) => {  
  const host = req.hostname;
```

```
const tenant = await db.tenants.findOne({
  where: {
    OR: [
      { subdomain: extractSubdomain(host) },
      { custom_domain: host }
    ]
  }
});

if (!tenant) {
  return res.status(404).send("Tenant not found");
}

req.tenant = tenant;

next();
});
```

6. NGINX Requirements

6.1 Wildcard Subdomain Support

NGINX must support:

*.syndicationx.com

Example

```
server {
  listen 80;

  server_name .syndicationx.com;
```



```
location / {  
    proxy_pass http://localhost:3000;  
  
    proxy_set_header Host $host;  
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;  
}  
}
```

6.2 Custom Domain Catch-All

```
server {  
    listen 80 default_server;  
    server_name _;  
  
    location / {  
        proxy_pass http://localhost:3000;  
  
        proxy_set_header Host $host;  
    }  
}
```

7. SSL Strategy

Phase 1 (Recommended Initially)

Wildcard SSL

For:

*.syndicationx.com

Use DNS challenge.

Benefits:

- easiest
 - stable
 - scalable
-

Phase 2

Per-Customer SSL

When custom domain added:

- generate certificate automatically
 - reload nginx
-

8. Domain Mapping Flow

User Flow

Step 1

Customer enters domain:

app.amazonmarketing.com

Step 2

System generates DNS instructions.

Example:

Create CNAME:

app.amazonmarketing.com

→ proxy.syndicationx.com

Step 3

Background worker verifies DNS.

Step 4

System issues SSL.

Step 5

NGINX reloads automatically.

9. Security Requirements

9.1 Tenant Isolation

Every query must include:

WHERE tenant_id = ?

9.2 Rate Limiting

Prevent abuse.

Use:

- Redis
 - NGINX rate limiting
-

9.3 DDoS Protection

Recommended:

- Cloudflare proxy
 - fail2ban
 - UFW firewall
-

9.4 Secret Management

Store:

- JWT secrets
- SMTP creds
- API keys

inside:

- Vault
 - environment variables
 - encrypted storage
-

10. Scalability Plan

Phase 1

Single VPS

NGINX

Node.js

Postgres

Redis

Phase 2

Separate DB server.

Phase 3

Load Balancer + Multiple App Nodes.

Phase 4

Kubernetes / Docker Swarm.

11. Deployment Requirements

CI/CD

Recommended:

- GitHub Actions
 - PM2 deploy
 - Docker
-

Deployment Flow

Git Push

→ Build

→ Test

→ Deploy

→ PM2 Reload

12. Monitoring

Use:

Tool	Purpose
PM2	Process monitoring
Grafana	Metrics
Prometheus	Server metrics
Loki	Logs
Sentry	Error tracking

13. Recommended Infrastructure

VPS Minimum

Resource	Recommended
----------	-------------

CPU	4 vCPU
-----	--------

RAM	8 GB
-----	------

SSD	100 GB
-----	--------

Bandwidth Unlimited preferred

14. Recommended Final Architecture

BEST PRACTICE

Cloudflare

|



NGINX Reverse Proxy

|



Node.js Multi-Tenant App

|



PostgreSQL + Redis

15. Critical Architectural Decisions

DO NOT

- ✗ Create separate Node.js app per tenant
- ✗ Share codebase with clients
- ✗ Use separate VPS per client initially
- ✗ Hardcode domains in NGINX manually

DO

- ✓ Single centralized app
- ✓ Dynamic tenant resolution
- ✓ Auto SSL provisioning
- ✓ DB-level tenant isolation
- ✓ Wildcard + custom domain support
- ✓ Cloudflare proxying

16. Recommended Folder Structure

/apps

/backend

/frontend

/nginx

/scripts

/certs

/docker

/services

tenant-service

ssl-service

domain-service

17. Suggested Future Features

- Tenant-specific API keys
- Usage billing
- White-label emails
- Tenant-level RBAC
- Multi-region deployment
- Edge caching
- CDN support
- Tenant analytics

18. Final Recommendation

For your case:

BEST INITIAL APPROACH

Use:

- Node.js multi-tenant backend
- NGINX reverse proxy
- PostgreSQL

- Redis
- Wildcard SSL
- Dynamic custom domain mapping
- Cloudflare
- PM2

This is the same architecture pattern used by many SaaS companies:

- Shopify
- Webflow
- Notion custom domains
- Vercel custom domains
- Ghost(Pro)

19. Success Criteria

The platform is considered successful when:

- ✓ amazon.syndicationx.com works automatically
- ✓ amazonmarketing.com works automatically
- ✓ SSL auto provisions
- ✓ Single deployment serves all tenants
- ✓ No source code exposure
- ✓ Tenant isolation enforced
- ✓ New tenants onboard in under 5 minutes
- ✓ Zero manual NGINX edits per customer

Next, I can help you create:

1. Full NGINX configuration architecture
2. Node.js tenant resolver implementation
3. SSL auto-provisioning system
4. Database schema design
5. Docker deployment architecture
6. Production-grade folder structure

7. Cloudflare setup guide
8. SaaS onboarding flow
9. Auto domain verification worker
10. Full DevOps architecture diagram
11. PM2 ecosystem config
12. Kubernetes-ready architecture
13. Security hardening checklist
14. Tenant middleware implementation
15. Multi-tenant PostgreSQL strategy